ECGR 4101/5101

Lab 8: Tic Tac Toe Game

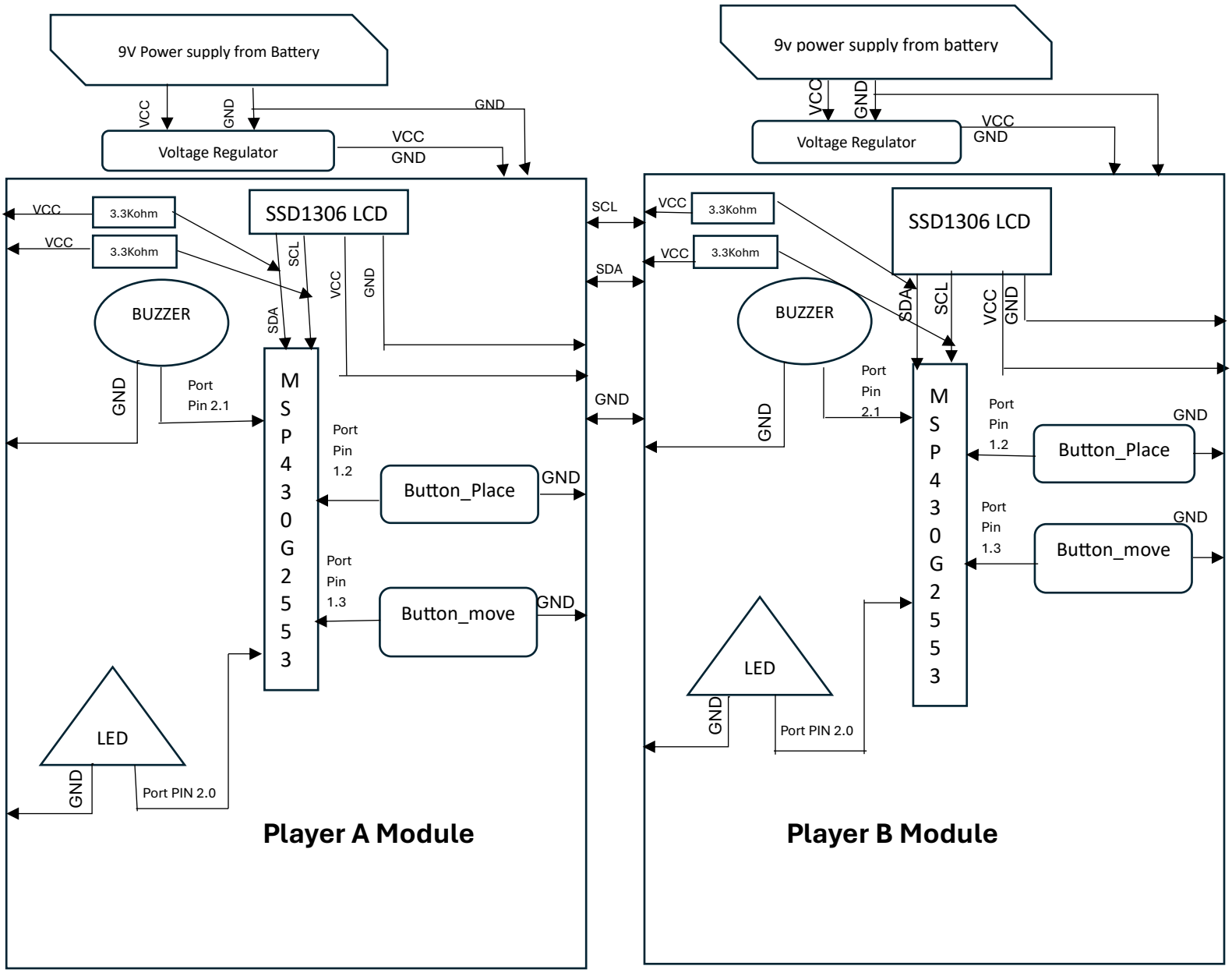
Hardware and Software architecture designs

Student Name: Poojitha Rajapuram

Student Name: Indhuja Gudluru

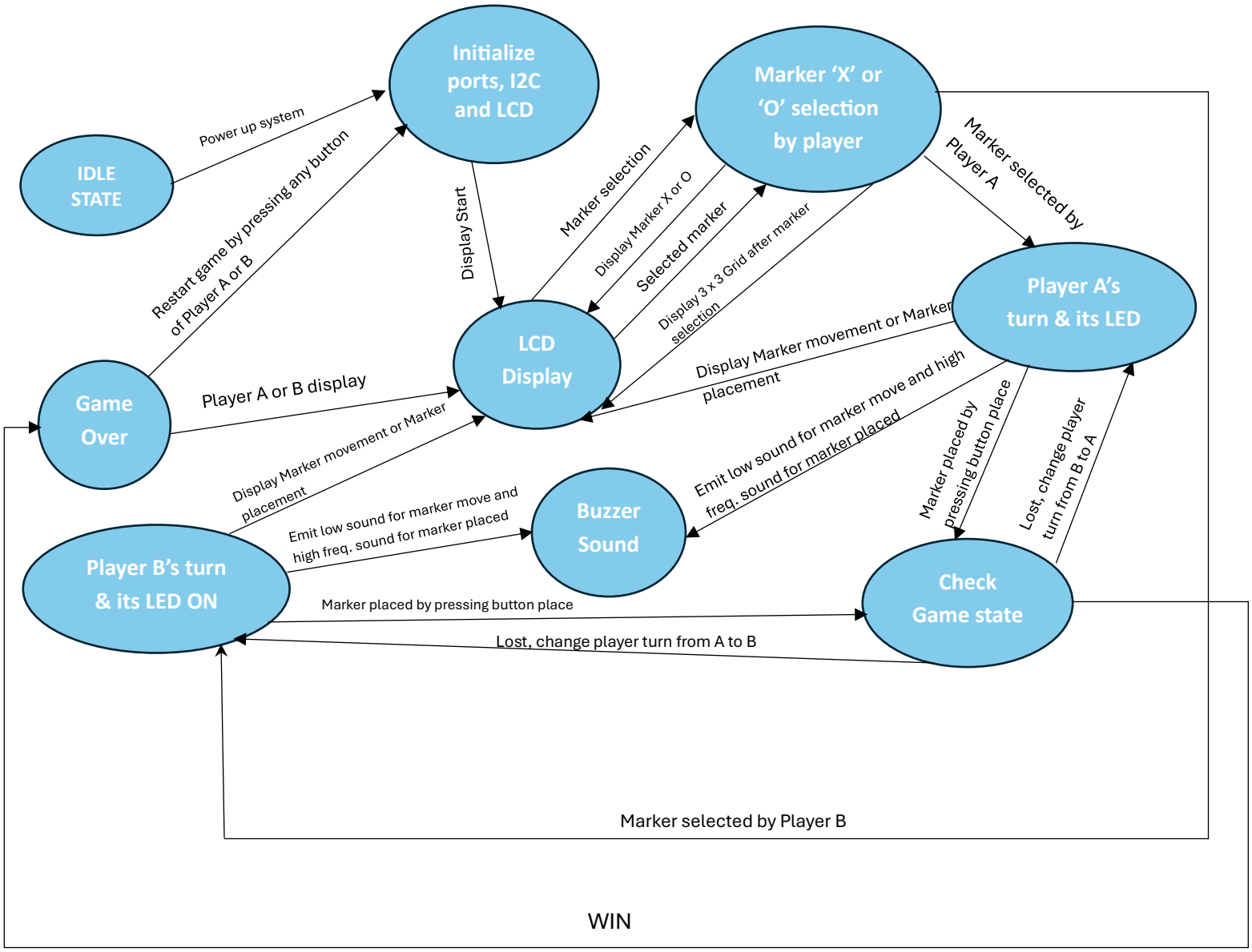
Date: 11/21/2024

Hardware Block Diagram



Software Architecture

State machine Diagram



Algorithms for Software Architecture

//Algorithm for Lab 8: Tic Tac Toe Game

//Authors: Poojitha Rajapuram, Indhuja Gudluru

//Version:1.1, Date: 11/21/2024

1. Main Function

1. Stop Watchdog Timer to prevent resets.
2. Call the initialization functions:
    - I2C setup.
    - LCD setup.
    - Buttons, LEDs, and buzzer setup.
3. Display Start Message:
    - On the LCD, display the initial message:
        - "START".
        - 3 x 3 grid display
4. Enter the infinite game loop:
    - Continuously wait for interrupts to handle player actions.
    - Perform periodic game state checks (driven by ISRs or function calls triggered by interrupts).

2. Initialization Functions

2.1 I2C Initialization

1. Set up I2C in Master/Slave mode:
    - Assign unique addresses for each microcontroller (e.g., 0x10 and 0x20).
2. Enable I2C interrupts for transmit, receive, and stop conditions
3. Configure I2C clock speed for smooth communication.

2.2 LCD Initialization

1. Configure LCD for I2C communication.
2. Clear the screen.
3. Set the cursor to the home position.

2.3 Button Initialization

1. Set up GPIO pins for navigation, mark placement, and reset buttons.
2. Enable interrupts on rising edge for each button pin.
3. Configure button debounce logic.

2.4 LED and Buzzer Initialization

1. Set up GPIO pins for the LED and buzzer.
2. Ensure the buzzer can generate low frequency and high frequency tones for navigation and marking actions respectively.

3. Display LCD messages Function

1. Clear the LCD.
2. Display:
    - "Start" for both players.
    - "Player A's Turn" for Player A's screen.
    - "Player B's Turn" for Player B's screen.
    - Player A win for Player A’s screen
    - Player B win for Player B’s screen
3. Ensure this message stays for 2 seconds.

4. Handle Player Turn Function

1. Wait for navigation button press (handled by ISR).
2. Update cursor position on the grid.
3. Wait for mark placement button press (handled by ISR).

4. Validate the chosen cell:
    - If valid: Place the mark (X or O) in the grid.
    - If invalid: Display error message and wait for another input.
5. Send the updated grid to the opponent via I2C.
6. Check for game state:
    - Win condition: Display result and blink LED.
    - Draw condition: Display "Draw!".
7. Switch to the next player's turn and update the LCD.

5. Game State Check Function

1. Check for three consecutive marks in:
    - Rows.
    - Columns.
    - Diagonals.
2. If a win condition is met:
    - Update game state to the winning player.
3. If no empty cells remain, set game state to "Draw."

6. Reset Game Function

1. Clear the 3x3 grid.
2. Reset the game state and current player.
3. Display the start message again.

7. Draw Grid Function

1. Clear the LCD.
2. Loop through the grid:

- Display the content of each cell (X, O, or blank).
- Add vertical bars (|) between cells and horizontal dividers (-+-+-) between rows.

3. Highlight the cursor's current position with an underscore (_).

8. Move Cursor

1. Detect the navigation button press direction.
2. Update the cursor position based on the direction:
   - Move up, down, left, or right with wrapping (e.g., top to bottom, left to right).
3. Play a navigation sound for feedback.
4. Refresh the grid display to show the new cursor position.

9. Place Cursor

1. Check if the current cell is empty:
   - If occupied, show "Invalid Move" on the LCD and return.
2. Place the current player's mark (X or O) in the cell.
3. Play a sound for placement.
4. Refresh the LCD to update the grid.
5. Check for win/draw conditions.
6. If game is not over, switch to the next player's turn.
7. Send the updated grid state via I2C.

10. Button ISR

10.1 Navigation Button ISR

1. Detect the button press event.
2. Update the cursor position:
   - Move the cursor to the next grid cell.

- Wrap around if the cursor reaches the edge of the grid.

3. Play a distinct buzzer sound for navigation feedback.
4. Refresh the LCD to show the updated cursor position.
5. Exit the ISR.

10.2. Mark Placement Button ISR

1. Detect the button press event.
2. Validate the selected grid cell:
    - If empty: Place the current player's mark (X or O) in the cell.
    - If occupied: Exit ISR and let the main loop handle the error message.
3. Update the LCD to show the updated grid.
4. Play a distinct buzzer sound for mark placement feedback.
5. Check the game state:
    - If a player wins, set the game state to "Win".
    - If no cells are empty, set the game state to "Draw".
6. Send the updated grid state to the opponent using I2C.
7. Switch the turn to the next player.
8. Exit the ISR.

11. I2C ISR Algorithm

11.1. I2C Transmit Interrupt (TXIFG)

1. Check if there is data to send:
    - If yes:
        - Load the next data byte into the I2C transmit buffer.
        - Increment the data pointer to prepare for the next byte.
    - If no more data to send:
        - Wait for the stop condition or complete transmission.

2. Exit the ISR.

11.2. I2C Receive Interrupt (RXIFG)

1. Read the received data from the I2C receive buffer.
2. Process the received data:
    - If the data represents a grid update:
        - Update the local 3x3 grid array with the opponent's move.
        - Refresh the LCD to show the updated grid.
    - If the data represents a game result (e.g., win/draw):
        - Update the game state and display the result on the LCD.
3. Exit the ISR.

11.3. I2C Stop Condition Interrupt (STPIFG)

1. Confirm the end of transmission or reception.
2. Reset any communication flags to prepare for the next operation.
3. Exit the ISR.